

Function

1. Quick Check: Prerequisite Knowledge

Before we build a “machine,” do you remember the parts?

- **Variables:** Storing data (e.g., `x = 5`).
- **Data Types:** Integers (`10`) and Strings (`"Hello"`).
- **Indentation:** The “space” at the start of a line that shows code belongs together.

2. Definition: Fill in the Blanks

A Python **function** is a reusable block of code. It only runs when you _____ it. You can pass data, known as _____, into it. A function can _____ data as a result.

(Answers: *call, parameters, return*)

3. Thinking Process: How to Build a Function

When you see a problem, follow these three steps:

1. **Identify the Input:** What data does the function need? (These go in the `()`).
2. **Define the Action:** What math or logic happens inside?
3. **Specify the Output:** What should the function give back? (Use `return`).

Example: The “Double It” Machine

- **Input:** A number (`$n$`).
- **Action:** `$n \times 2$`.
- **Output:** The result.

```
def double_number(n):  
    result = n * 2  
    return result
```

4. Practice 1: Scaffolding (Easy)

Complete the function to add two numbers together.

```
def add_numbers(a, b):  
    # Your code here  
    answer = ____ + ____  
    return _____
```

```
# Test it:
print(add_numbers(5, 10))
```

5. Practice 2: Advancing (Applying)

Create a function called `make_full_name` . It should take a `first_name` and a `last_name` , combine them with a space, and return the result.

Goal: `make_full_name("Bruce", "Wayne")` should return `"Bruce Wayne"` .

6. Exercise Set: The Challenge Loop

Task	Level	Goal
The Calculator	Basic	Create a function <code>subtract(a, b)</code> that returns <code>\$a - b\$</code> .
The Messenger	Intermediate	Create a function that takes a name and returns <code>"Hello, [name]!"</code> .
The Converter	Advanced	Create a function <code>to_seconds(minutes)</code> that converts minutes to seconds (<code>\$minutes \times 60\$</code>).